# OS Practical Slips – Corrected Codes & Viva Questions

Oct 13, 2026 · @Faisal

## Overview

All 11 slips are below, each with its corrected, exam-ready code and the viva questions an external examiner is likely to ask. Every program here compiles with gcc on Linux and follows the slip's question.

Write every printf string and every // comment on ONE line. The original PDF broke several of them across lines, which causes compile errors.

| Slip | Q.1 | Q.2 | Expected answer for the slip's data |
|---|---|---|---|
| 1 | Banker's (menu) | Shell with count | Depends on snapshot |
| 2 | Banker's (safety + request) | SSTF | SSTF total = 232 |
| 3 | Banker's (menu, 7 2 6 instances) | SCAN | SCAN total = 240 |
| 4 | Optimal page replacement | Non-preemptive SJF | Optimal = 9 faults |
| 5 | Round Robin | LOOK | LOOK total = 224 |
| 6 | Banker's (menu) | Optimal page replacement | Optimal = 9 faults |
| 7 | FCFS | Fork: bubble + insertion sort | Depends on input |
| 8 | Non-preemptive SJF | LOOK | LOOK total = 240 |
| 9 | LRU (counter) | Shell with count | LRU = 9 faults |
| 10 | Orphan process | Shell with search | Depends on file |
| 11 | SCAN | MFU page replacement | SCAN = 260, MFU = 10 faults |

Assumptions: 3 frames for page replacement, and disk size 200 (blocks 0 to 199) for disk scheduling. Compile with `gcc prog.c -o prog` and run with `./prog`.

Common formulas to remember: TAT = CT - AT, WT = TAT - BT, Need = Max - Allocation, Available = Total - column sum of Allocation.

## Slip 1

### Q.1 Banker's Algorithm – Menu Driven

Accept Allocation, Max and Available, then calculate and display Need. Enter the options in the order 1, 2, 4, 3, 5. If you display before calculating, Need shows zeros.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10

int max[MAX][MAX], need[MAX][MAX], allocation[MAX][MAX];
int available[MAX];
int np, nr;

void accept_matrix(int A[MAX][MAX])
{
    int i, j;
    for (i = 0; i < np; i++)
    {
        printf("P%d: ", i);
        for (j = 0; j < nr; j++)
            scanf("%d", &A[i][j]);
    }
}

void accept_vector(int A[MAX])
{
    int i;
    for (i = 0; i < nr; i++)
        scanf("%d", &A[i]);
}

void calculate_need()
{
    int i, j;
    for (i = 0; i < np; i++)
        for (j = 0; j < nr; j++)
            need[i][j] = max[i][j] - allocation[i][j];
    printf("\nNeed matrix calculated successfully.\n");
}

void display()
{
    int i, j;
    printf("\nProcess\tAllocation\tMax\t\tNeed\n");
    for (i = 0; i < np; i++)
    {
        printf("P%d\t", i);
        for (j = 0; j < nr; j++) printf("%d ", allocation[i][j]);
        printf("\t\t");
        for (j = 0; j < nr; j++) printf("%d ", max[i][j]);
        printf("\t\t");
        for (j = 0; j < nr; j++) printf("%d ", need[i][j]);
        printf("\n");
```

```
    }
    printf("\nAvailable: ");
    for (i = 0; i < nr; i++) printf("%d ", available[i]);
    printf("\n");
}

int main()
{
    int choice;
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter number of resources: ");
    scanf("%d", &nr);

    do
    {
        printf("\n----- MENU -----\n");
        printf("1. Accept Allocation\n");
        printf("2. Accept Max\n");
        printf("3. Calculate Need\n");
        printf("4. Accept Available\n");
        printf("5. Display Matrices\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: printf("Enter Allocation Matrix:\n"); accept_matrix(allocation);
break;
            case 2: printf("Enter Max Matrix:\n"); accept_matrix(max); break;
            case 3: calculate_need(); break;
            case 4: printf("Enter Available Vector:\n"); accept_vector(available);
break;
            case 5: display(); break;
            case 6: break;
            default: printf("Invalid choice\n");
        }
    } while (choice != 6);
    return 0;
}
```

**Viva questions – Banker's Algorithm**

1. What is a deadlock? A set of processes where each waits for a resource held by another, so none can proceed.
2. What are the four necessary conditions? Mutual exclusion, hold and wait, no preemption and circular wait.
3. What are the ways to handle deadlock? Prevention, avoidance (Banker's), detection and recovery, and ignoring it (the Ostrich approach used by Linux and Windows).
4. Why is it called Banker's algorithm? A bank never lends in a way that leaves it unable to satisfy all customers. The OS allocates resources the same way.
5. Which data structures are used? Available[m], Max[n][m], Allocation[n][m] and Need[n][m].
6. What is the formula for Need? Need[i][j] = Max[i][j] - Allocation[i][j].

7. What does Available represent? The free instances of each resource type.
8. What happens if you display before calculating Need? Need shows 0, because global arrays are initialised to 0.
9. Why are the arrays global? Every function can use them without passing them, and they start at 0.
10. What does #define MAX 10 do? It is a macro for the maximum number of processes and resources.
11. How is a 2D array passed to a function? As int A[MAX][MAX]. The column size must be given.
12. Why use do-while for the menu? The menu must be shown at least once.
13. What if Max is less than Allocation? Need becomes negative, which means the input is invalid.
14. What are the limitations of Banker's? Max need must be known in advance, the number of processes is fixed, and the cost is O(m x n x n).
15. How does prevention differ from avoidance? Prevention breaks one of the four conditions. Avoidance keeps the system in a safe state dynamically.
16. What is a Resource Allocation Graph? A graph of processes and resources. A cycle is necessary for deadlock, and sufficient only with single-instance resources.

### Q.2 Shell with the count command

Displays a $ prompt, tokenizes the command, runs it in a child process, and supports count c, count w and count l.

```
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void make_toks(char *s, char *tok[])
{
    int i = 0;
    char *p = strtok(s, " \t");
    while (p != NULL)
    {
        tok[i++] = p;
        p = strtok(NULL, " \t");
    }
    tok[i] = NULL;
}

void count(char *fn, char op)
{
    int fh, cc = 0, wc = 0, lc = 0, inword = 0;
    char c;
    fh = open(fn, O_RDONLY);
    if (fh == -1)
    {
        printf("File %s not found.\n", fn);
        return;
```

```
    }
    while (read(fh, &c, 1) > 0)
    {
        cc++;
        if (c == '\n') lc++;
        if (c == ' ' || c == '\t' || c == '\n') inword = 0;
        else if (!inword) { inword = 1; wc++; }
    }
    close(fh);
    switch (op)
    {
        case 'c': printf("No. of characters: %d\n", cc); break;
        case 'w': printf("No. of words: %d\n", wc); break;
        case 'l': printf("No. of lines: %d\n", lc); break;
        default : printf("Invalid option. Use c, w or l\n");
    }
}

int main()
{
    char buff[80], *args[10];
    pid_t pid;

    while (1)
    {
        printf("$ ");
        fflush(stdout);
        if (fgets(buff, 80, stdin) == NULL) break;
        buff[strcspn(buff, "\n")] = '\0';
        make_toks(buff, args);
        if (args[0] == NULL) continue;

        if (strcmp(args[0], "quit") == 0)
            break;
        else if (strcmp(args[0], "count") == 0)
        {
            if (args[1] == NULL || args[2] == NULL)
                printf("Usage: count c|w|l filename\n");
            else
                count(args[2], args[1][0]);
        }
        else
        {
            pid = fork();
            if (pid > 0)
                wait(NULL);
            else if (pid == 0)
            {
                if (execvp(args[0], args) == -1)
                {
                    printf("Bad command.\n");
                    exit(1);
                }
            }
            else
```

```
                printf("Fork failed\n");
        }
    }
    return 0;
}
```

Fixes made: the child now exits if execvp fails, an empty line and missing arguments no longer crash it, fflush(stdin) is removed, and word counting handles multiple spaces and tabs.

**Viva questions – Shell**

1. What is a shell? A command interpreter between the user and the kernel, such as bash or sh.
2. What does fork() return? 0 in the child, the child's PID in the parent, and -1 on failure.
3. What does execvp() do? It replaces the current process image with a new program. v means an argument array, p means it searches PATH.
4. Why fork before exec? exec replaces the caller. Without fork, the shell itself would be replaced and would end.
5. Why does the parent call wait()? To wait for the child to finish and avoid a zombie process.
6. What happens after a successful exec? It never returns. Code after it runs only on failure.
7. Why must args end with NULL? execvp needs a NULL-terminated argument list.
8. What does strtok() do? It splits a string into tokens. Later calls pass NULL to continue on the same string.
9. Why remove the newline from buff? fgets keeps the newline, which would become part of the last argument.
10. Why exit(1) after a failed execvp? Otherwise the child continues as a second shell.
11. What is a zombie process? A finished child whose parent has not called wait(), so its process table entry remains.
12. Why can't cd be run with exec? A child cannot change the parent's directory, so cd must be built in using chdir().
13. How does open() differ from fopen()? open is a system call returning an int file descriptor. fopen is a library function returning a buffered FILE pointer.
14. What does read() return? Bytes read, 0 at end of file, and -1 on error.
15. What are file descriptors 0, 1 and 2? stdin, stdout and stderr.
16. How does a system call differ from a library function? A system call switches to kernel mode. A library function runs in user mode.
17. Which Linux command does count imitate? wc -c, wc -w and wc -l.
18. How are words counted? A word starts whenever a non-space character follows a space, tab or newline.

## Slip 3

### Q.1 Banker's Algorithm – Need Matrix and Safe Sequence

Run the options in order 1, 2, 4, 3, then 6 for the safe sequence. Option 7 tests a resource request.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10
```

```
int max[MAX][MAX], need[MAX][MAX], allocation[MAX][MAX];
int available[MAX], work[MAX], finish[MAX], request[MAX], safeSeq[MAX];
int np, nr;

void accept_matrix(int A[MAX][MAX])
{
    int i, j;
    for (i = 0; i < np; i++)
    {
        printf("P%d: ", i);
        for (j = 0; j < nr; j++)
            scanf("%d", &A[i][j]);
    }
}

void accept_vector(int A[MAX])
{
    int i;
    for (i = 0; i < nr; i++)
        scanf("%d", &A[i]);
}

void calculate_need()
{
    int i, j;
    for (i = 0; i < np; i++)
        for (j = 0; j < nr; j++)
            need[i][j] = max[i][j] - allocation[i][j];
    printf("\nNeed matrix calculated successfully.\n");
}

void display()
{
    int i, j;
    printf("\nProcess\tAllocation\tMax\t\tNeed\n");
    for (i = 0; i < np; i++)
    {
        printf("P%d\t", i);
        for (j = 0; j < nr; j++) printf("%d ", allocation[i][j]);
        printf("\t\t");
        for (j = 0; j < nr; j++) printf("%d ", max[i][j]);
        printf("\t\t");
        for (j = 0; j < nr; j++) printf("%d ", need[i][j]);
        printf("\n");
    }
    printf("\nAvailable: ");
    for (i = 0; i < nr; i++) printf("%d ", available[i]);
    printf("\n");
}

int safety_algorithm()
{
    int i, j, count = 0, found, flag;
    for (i = 0; i < nr; i++) work[i] = available[i];
    for (i = 0; i < np; i++) finish[i] = 0;
```

```
    while (count < np)
    {
        found = 0;
        for (i = 0; i < np; i++)
        {
            if (finish[i] == 0)
            {
                flag = 0;
                for (j = 0; j < nr; j++)
                    if (need[i][j] > work[j]) { flag = 1; break; }
                if (flag == 0)
                {
                    for (j = 0; j < nr; j++) work[j] += allocation[i][j];
                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if (found == 0) return 0;    /* unsafe */
    }
    return 1;                        /* safe */
}

void bankers_algorithm()
{
    int i;
    if (safety_algorithm())
    {
        printf("\nSystem is in SAFE state.\nSafe Sequence: ");
        for (i = 0; i < np; i++) printf("P%d ", safeSeq[i]);
        printf("\n");
    }
    else
        printf("\nSystem is in UNSAFE state (deadlock may occur).\n");
}

void resource_request()
{
    int i, pno;
    printf("Enter process number: ");
    scanf("%d", &pno);
    if (pno < 0 || pno >= np) { printf("Invalid process\n"); return; }
    printf("Enter request vector: ");
    accept_vector(request);

    for (i = 0; i < nr; i++)
        if (request[i] > need[pno][i]) { printf("Error: request exceeds maximum
need.\n"); return; }
    for (i = 0; i < nr; i++)
        if (request[i] > available[i]) { printf("Resources not available. Process
must wait.\n"); return; }

    for (i = 0; i < nr; i++)          /* pretend to allocate */
```

```
    {
        available[i] -= request[i];
        allocation[pno][i] += request[i];
        need[pno][i] -= request[i];
    }
    if (safety_algorithm())
        printf("Request granted. System remains SAFE.\n");
    else
    {
        printf("Request cannot be granted. System becomes UNSAFE.\n");
        for (i = 0; i < nr; i++)      /* rollback */
        {
            available[i] += request[i];
            allocation[pno][i] -= request[i];
            need[pno][i] += request[i];
        }
    }
}

int main()
{
    int choice;
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter number of resources: ");
    scanf("%d", &nr);

    do
    {
        printf("\n----- MENU -----\n");
        printf("1. Accept Allocation\n2. Accept Max\n3. Calculate Need\n");
        printf("4. Accept Available\n5. Display Matrices\n6. Apply Banker's
Algorithm\n");
        printf("7. Resource Request Algorithm\n8. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1: printf("Enter Allocation Matrix:\n"); accept_matrix(allocation);
break;
            case 2: printf("Enter Max Matrix:\n"); accept_matrix(max); break;
            case 3: calculate_need(); break;
            case 4: printf("Enter Available Vector:\n"); accept_vector(available);
break;
            case 5: display(); break;
            case 6: bankers_algorithm(); break;
            case 7: resource_request(); break;
            case 8: break;
            default: printf("Invalid choice\n");
        }
    } while (choice != 8);
    return 0;
}
```

**Viva questions – Safety algorithm** (also revise the Banker's questions in Slip 1)

1. What is a safe state? A state with at least one order (safe sequence) in which every process can finish.
2. Does an unsafe state always mean deadlock? No. It may lead to deadlock. Every deadlock state is unsafe.
3. What are the steps of the safety algorithm? Work = Available, Finish = false. Find i with Finish false and Need <= Work, then Work += Allocation[i] and Finish[i] = true. Repeat. All finished means safe.
4. What are Work and Finish? Work is the simulated free resources. Finish marks processes that can complete.
5. Is the safe sequence unique? No. The program prints the first one found while scanning P0 to Pn.
6. What are the steps of the resource-request algorithm? Check Request <= Need, then Request <= Available, pretend to allocate, run the safety check, and roll back if unsafe.
7. Why roll back? Granting would make the system unsafe, so the process waits and the old state is restored.
8. What is the found flag for? If a full pass finds no runnable process, the state is unsafe, and the loop must stop.
9. What is the time complexity? O(m x n x n).
10. How does deadlock detection differ? It uses a Request matrix in place of Need and runs after allocation.
11. How do you recover from deadlock? Terminate processes, or preempt resources by choosing a victim and rolling it back.
12. Why must Need be calculated before option 6? Otherwise Need is 0 and every system looks safe.

### Q.2 SSTF Disk Scheduling

Input: 98 183 37 122 14 124 65, head at 53. Expected order: 53, 65, 37, 14, 98, 122, 124, 183. Total head movement = 232.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int rq[100], visited[100] = {0};
    int n, i, j, size, head, idx, total = 0;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &size);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &rq[i]);
        if (rq[i] < 0 || rq[i] >= size)
        {
            printf("Invalid request, enter again: ");
```

```
            i--;
        }
    }
    printf("Enter current head position: ");
    scanf("%d", &head);

    printf("\nOrder of service: %d", head);
    for (i = 0; i < n; i++)
    {
        idx = -1;
        for (j = 0; j < n; j++)       /* nearest unvisited request */
            if (!visited[j] && (idx == -1 || abs(rq[j] - head) < abs(rq[idx] -
head)))
                idx = j;
        visited[idx] = 1;
        total += abs(rq[idx] - head);
        head = rq[idx];
        printf(" -> %d", head);
    }
    printf("\nTotal head movement = %d\n", total);
    return 0;
}
```

**Viva questions – Disk scheduling (common to all disk slips)**

1. What is seek time? The time to move the head to the required track.
2. What is rotational latency? The time for the required sector to rotate under the head.
3. What is transfer time? The time to actually read or write the data.
4. Which disk scheduling algorithms exist? FCFS, SSTF, SCAN, C-SCAN, LOOK and C-LOOK.
5. What is the goal of disk scheduling? To minimise total head movement and improve throughput.
6. What is SSTF? Shortest Seek Time First: always serve the request nearest to the head.
7. What is the drawback of SSTF? Starvation of requests far from the head.
8. Is SSTF optimal? No. It is greedy and gives a good average, not always the minimum.
9. Why the visited array? To mark requests already served.
10. What does abs() do, and where is it declared? It gives the absolute difference and is in stdlib.h.
11. Which CPU algorithm is SSTF similar to? SJF.
12. Show the manual calculation. 12 + 28 + 23 + 84 + 24 + 2 + 59 = 232.
13. Which algorithm is best in practice? SSTF or LOOK for normal load, and C-SCAN or C-LOOK under heavy load for uniform waiting.

## Slip 4

### Q.1 Banker's Algorithm – Menu (A, B, C with 7, 2, 6 instances)

The slip gives total instances, so option 4 calculates Available = Total - column sum of Allocation. Enter Allocation first, then Max, then option 4 with totals 7 2 6.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10

int max[MAX][MAX], need[MAX][MAX], allocation[MAX][MAX];
```

```
int available[MAX], total[MAX];
int np, nr;

void accept_matrix(int A[MAX][MAX])
{
    int i, j;
    for (i = 0; i < np; i++)
    {
        printf("P%d: ", i);
        for (j = 0; j < nr; j++)
            scanf("%d", &A[i][j]);
    }
}

void accept_vector(int A[MAX])
{
    int i;
    for (i = 0; i < nr; i++)
        scanf("%d", &A[i]);
}

void calculate_available()
{
    int i, j;
    printf("Enter total instances of each resource (e.g. 7 2 6): ");
    accept_vector(total);
    for (j = 0; j < nr; j++)
    {
        available[j] = total[j];
        for (i = 0; i < np; i++)
            available[j] -= allocation[i][j];
    }
    printf("Available calculated as Total - sum of Allocation.\n");
}

void calculate_need()
{
    int i, j;
    for (i = 0; i < np; i++)
        for (j = 0; j < nr; j++)
            need[i][j] = max[i][j] - allocation[i][j];
    printf("\nNeed matrix calculated successfully.\n");
}

void display()
{
    int i, j;
    printf("\nProcess\tAllocation\tMax\t\tNeed\n");
    for (i = 0; i < np; i++)
    {
        printf("P%d\t", i);
        for (j = 0; j < nr; j++) printf("%d ", allocation[i][j]);
        printf("\t\t");
        for (j = 0; j < nr; j++) printf("%d ", max[i][j]);
        printf("\t\t");
```

```
        for (j = 0; j < nr; j++) printf("%d ", need[i][j]);
        printf("\n");
    }
    printf("\nAvailable: ");
    for (i = 0; i < nr; i++) printf("%d ", available[i]);
    printf("\n");
}

int main()
{
    int choice;
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter number of resources: ");
    scanf("%d", &nr);

    do
    {
        printf("\n----- MENU -----\n");
        printf("1. Accept Allocation\n2. Accept Max\n3. Calculate Need\n");
        printf("4. Calculate Available from total instances\n5. Display Matrices\n6.
Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1: printf("Enter Allocation Matrix:\n"); accept_matrix(allocation);
break;
            case 2: printf("Enter Max Matrix:\n"); accept_matrix(max); break;
            case 3: calculate_need(); break;
            case 4: calculate_available(); break;
            case 5: display(); break;
            case 6: break;
            default: printf("Invalid choice\n");
        }
    } while (choice != 6);
    return 0;
}
```

**Viva questions:** all the Banker's questions from Slip 1, plus:

1. How do you get Available from total instances? Available = Total - column sum of Allocation.
2. What do 7, 2, 6 instances mean? A has 7 copies, B has 2 and C has 6 in the whole system.
3. What if the column sum of Allocation is more than Total? The input is invalid, since you cannot allocate more than exists.
4. Why must Allocation be entered before option 4? Available is calculated from Allocation.

### Q.2 SCAN Disk Scheduling

Input: 82 170 43 140 24 16 190 65, head 50, direction Left, disk size 200. Expected order: 50, 43, 24, 16, 0, 65, 82, 140, 170, 190. Total = 50 + 190 = 240.

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int rq[100], n, i, j, temp, size, head, dir, idx, cur, total = 0;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &size);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string:\n");
    for (i = 0; i < n; i++) scanf("%d", &rq[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)                 /* bubble sort */
        for (j = 0; j < n - i - 1; j++)
            if (rq[j] > rq[j + 1])
            {
                temp = rq[j]; rq[j] = rq[j + 1]; rq[j + 1] = temp;
            }

    idx = n;                                    /* first request greater than head */
    for (i = 0; i < n; i++)
        if (rq[i] > head) { idx = i; break; }

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0)                               /* LEFT */
    {
        for (i = idx - 1; i >= 0; i--)
        {
            printf(" -> %d", rq[i]); total += cur - rq[i]; cur = rq[i];
        }
        printf(" -> 0"); total += cur; cur = 0;  /* go to the end of the disk */
        for (i = idx; i < n; i++)
        {
            printf(" -> %d", rq[i]); total += rq[i] - cur; cur = rq[i];
        }
    }
    else                                        /* RIGHT */
    {
        for (i = idx; i < n; i++)
        {
            printf(" -> %d", rq[i]); total += rq[i] - cur; cur = rq[i];
        }
        printf(" -> %d", size - 1); total += size - 1 - cur; cur = size - 1;
        for (i = idx - 1; i >= 0; i--)
        {
            printf(" -> %d", rq[i]); total += cur - rq[i]; cur = rq[i];
        }
    }
    printf("\nTotal head movement = %d\n", total);
    return 0;
}
```

**Viva questions – SCAN** (also revise the disk questions in Slip 2)

1. What is SCAN? The head moves in one direction to the end of the disk, serving requests on the way, then reverses.
2. Why is SCAN called the elevator algorithm? It moves like a lift going up to the top floor and then coming down.
3. How does C-SCAN differ? It returns to the start without serving on the way back, giving more uniform waiting time.
4. How does LOOK differ? LOOK reverses at the last request. SCAN goes all the way to the disk end.
5. Why sort the requests? To serve them in increasing or decreasing order along the direction.
6. Why go to 0 when moving left? The definition of SCAN requires reaching the end of the disk.
7. What is the shortcut for Left? Head + largest request = 50 + 190 = 240.
8. Does SCAN starve requests? No. Every request is served within two sweeps.
9. What does idx store? The position of the first request greater than the head, which splits left and right requests.

## Slip 9

### Q.1 Optimal Page Replacement

Reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2 (15 pages). With 3 frames, total page faults = 9.

```
#include <stdio.h>

int main()
{
    int no_of_frames, no_of_pages, frames[10], pages[30], temp[10];
    int found, freeSlot, neverUsed, i, j, k, pos, max, faults = 0;

    printf("Enter number of frames: ");
    scanf("%d", &no_of_frames);
    printf("Enter number of pages: ");
    scanf("%d", &no_of_pages);
    printf("Enter page reference string: ");
    for (i = 0; i < no_of_pages; ++i)
        scanf("%d", &pages[i]);

    for (i = 0; i < no_of_frames; ++i)
        frames[i] = -1;                         /* -1 = empty frame */

    printf("\nPage Replacement Process:\n");
    for (i = 0; i < no_of_pages; ++i)
    {
        found = freeSlot = 0;
        for (j = 0; j < no_of_frames; ++j)     /* page already present? */
            if (frames[j] == pages[i]) { found = 1; break; }

        if (!found)                             /* use an empty frame */
        {
            for (j = 0; j < no_of_frames; ++j)
                if (frames[j] == -1)
                {
```

```
                    frames[j] = pages[i];
                    faults++;
                    freeSlot = 1;
                    break;
                }
        }

        if (!found && !freeSlot)              /* replace using future knowledge */
        {
            for (j = 0; j < no_of_frames; ++j)
            {
                temp[j] = -1;
                for (k = i + 1; k < no_of_pages; ++k)
                    if (frames[j] == pages[k]) { temp[j] = k; break; }
            }
            neverUsed = 0;
            for (j = 0; j < no_of_frames; ++j)
                if (temp[j] == -1) { pos = j; neverUsed = 1; break; }
            if (!neverUsed)
            {
                max = temp[0]; pos = 0;
                for (j = 1; j < no_of_frames; ++j)
                    if (temp[j] > max) { max = temp[j]; pos = j; }
            }
            frames[pos] = pages[i];
            faults++;
        }

        printf("Page %2d -> ", pages[i]);
        for (j = 0; j < no_of_frames; ++j)
        {
            if (frames[j] != -1) printf("%d ", frames[j]);
            else printf("- ");
        }
        printf("%s\n", found ? "(Hit)" : "(Fault)");
    }
    printf("\nTotal Page Faults = %d\n", faults);
    printf("Total Page Hits = %d\n", no_of_pages - faults);
    return 0;
}
```

**Viva questions – Page replacement (common to all paging slips)**

1. What is paging? Logical memory is divided into pages and physical memory into frames of the same size.
2. What is demand paging? A page is loaded only when it is needed.
3. What is a page fault? A reference to a page that is not in memory.
4. What is page replacement? Choosing which page to remove when all frames are full.
5. What is the Optimal algorithm? It replaces the page that will not be used for the longest time in the future.
6. Why can't a real OS implement Optimal? It needs knowledge of future references.
7. Then why study it? It gives the minimum possible faults and is used as a benchmark.

8. What is Belady's anomaly? More frames giving more faults. It occurs in FIFO, not in Optimal or LRU (stack algorithms).
9. What does frames[i] = -1 mean? The frame is empty.
10. What is temp[] for? The next-use position of each page in memory. -1 means never used again.
11. What is the hit ratio? Hits divided by total references. Here 6 / 15 = 0.4.
12. What is thrashing? The system spends more time swapping pages than executing, due to too few frames.
13. What are the valid-invalid bit and TLB? The bit shows whether a page is in memory. The TLB is a fast cache of page-table entries.
14. What is a page table? A table mapping each page number to a frame number.

### Q.2 Non-preemptive SJF Scheduling

At each step the CPU picks the shortest burst among the processes that have arrived. The next CPU burst is generated randomly.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int n, i, t = 0, done = 0, k = 0, idx;
    int at[10], bt[10], ct[10], tat[10], wt[10], fin[10] = {0};
    int order[10], st[10];
    float twt = 0, ttat = 0;

    srand(time(NULL));
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("Arrival time and first CPU burst of P%d: ", i);
        scanf("%d %d", &at[i], &bt[i]);
    }

    while (done < n)
    {
        idx = -1;
        for (i = 0; i < n; i++)               /* shortest job among arrived ones */
            if (!fin[i] && at[i] <= t)
                if (idx == -1 || bt[i] < bt[idx] || (bt[i] == bt[idx] && at[i] <
at[idx]))
                    idx = i;
        if (idx == -1) { t++; continue; }     /* nobody arrived: CPU idle */

        st[k] = t;
        order[k++] = idx;
        t += bt[idx];
        ct[idx] = t;
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];
        twt += wt[idx];
```

```
        ttat += tat[idx];
        fin[idx] = 1;
        done++;
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i, at[i], bt[i], ct[i], tat[i], wt[i]);

    printf("\nGantt Chart\n");
    for (i = 0; i < k; i++) printf("|  P%d  ", order[i]);
    printf("|\n");
    for (i = 0; i < k; i++) printf("%-7d", st[i]);
    printf("%d\n", t);

    printf("\nAverage Waiting Time = %.2f\n", twt / n);
    printf("Average Turnaround Time = %.2f\n", ttat / n);

    printf("\nNext CPU burst (generated randomly):\n");
    for (i = 0; i < n; i++)
        printf("P%d : %d\n", i, rand() % 10 + 1);
    return 0;
}
```

**Viva questions – CPU scheduling (common to all scheduling slips)**

1. What are AT, BT, CT, TAT and WT? Arrival, Burst, Completion, Turnaround and Waiting time.
2. What are the formulas? TAT = CT - AT and WT = TAT - BT.
3. What is response time? First time on the CPU minus arrival time.
4. What is the difference between preemptive and non-preemptive? Preemptive can take the CPU away from a running process. Non-preemptive lets it finish its burst.
5. What is SJF? It picks the process with the smallest burst among those that have arrived.
6. Why is SJF optimal? It gives the minimum average waiting time.
7. What are SJF's problems? Starvation of long jobs, and burst times are not known in advance.
8. What is the preemptive form of SJF? SRTF, Shortest Remaining Time First.
9. How does the OS predict the next burst? Exponential averaging: next = a x actual + (1 - a) x previous prediction.
10. Why is the next burst generated randomly here? To simulate an unknown future burst using rand() % 10 + 1.
11. Why srand(time(NULL))? To get different random numbers on every run.
12. What if no process has arrived? The CPU stays idle and time increases by 1.
13. What is a Gantt chart? A timeline showing which process runs in which interval.
14. How is starvation removed? Aging: priority increases with waiting time.
15. What are a context switch and the dispatcher? A context switch saves and loads process state. The dispatcher gives the CPU to the chosen process.
16. How are ties in burst time broken? The process that arrived earlier runs first.
17. What is the fin[] array? It marks processes that have completed.

## Slip 10

### Q.1 Round Robin Scheduling

Accepts arrival time, first CPU burst and time quantum. It uses a ready queue, so processes arriving later are handled correctly.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int n, tq, i, p, run, t = 0, done = 0;
    int at[10], bt[10], rem[10], ct[10], tat[10], wt[10], added[10] = {0};
    int q[500], front = 0, rear = 0;
    int gp[500], gs[500], g = 0;
    float twt = 0, ttat = 0;

    srand(time(NULL));
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter time quantum: ");
    scanf("%d", &tq);
    for (i = 0; i < n; i++)
    {
        printf("Arrival time and first CPU burst of P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i];
    }

    for (i = 0; i < n; i++)                    /* processes arriving at time 0 */
        if (at[i] <= t) { q[rear++] = i; added[i] = 1; }

    while (done < n)
    {
        if (front == rear)                     /* ready queue empty: CPU idle */
        {
            t++;
            for (i = 0; i < n; i++)
                if (!added[i] && at[i] <= t) { q[rear++] = i; added[i] = 1; }
            continue;
        }
        p = q[front++];
        run = (rem[p] < tq) ? rem[p] : tq;
        gp[g] = p;
        gs[g++] = t;
        t += run;
        rem[p] -= run;

        for (i = 0; i < n; i++)                /* new arrivals join before preempted
one */
            if (!added[i] && at[i] <= t) { q[rear++] = i; added[i] = 1; }

        if (rem[p] > 0)
```

```
            q[rear++] = p;                      /* not finished: back of queue */
        else
        {
            ct[p] = t;
            tat[p] = ct[p] - at[p];
            wt[p] = tat[p] - bt[p];
            twt += wt[p];
            ttat += tat[p];
            done++;
        }
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i],
wt[i]);

    printf("\nGantt Chart\n");
    for (i = 0; i < g; i++) printf("|  P%d  ", gp[i] + 1);
    printf("|\n");
    for (i = 0; i < g; i++) printf("%-7d", gs[i]);
    printf("%d\n", t);

    printf("\nAverage Waiting Time = %.2f\n", twt / n);
    printf("Average Turnaround Time = %.2f\n", ttat / n);

    printf("\nNext CPU burst (generated randomly):\n");
    for (i = 0; i < n; i++)
        printf("P%d : %d\n", i + 1, rand() % 10 + 1);
    return 0;
}
```

**Viva questions – Round Robin** (also revise the CPU scheduling questions in Slip 4)

1. What is Round Robin? Each process gets the CPU for a fixed time quantum in circular FIFO order.
2. Is RR preemptive? Yes. The process is preempted when its quantum expires.
3. What if the quantum is very large? RR becomes FCFS.
4. What if the quantum is very small? Too many context switches, so overhead is high.
5. How is the quantum chosen? About 80 percent of CPU bursts should be shorter than it.
6. What are RR's advantages? Fairness, good response time and no starvation. It suits time-sharing systems.
7. What is RR's disadvantage? Higher average waiting time than SJF.
8. What is rem[]? The remaining burst time of each process.
9. Why are new arrivals added before the preempted process? That is the standard RR ready-queue convention.
10. What happens when remaining time is less than the quantum? The process runs only for its remaining time and finishes.
11. What is added[] for? It ensures each process enters the ready queue only once on arrival.
12. What are gp[] and gs[]? The process and start time of each Gantt chart slot.

### Q.2 LOOK Disk Scheduling

Input: 55 58 39 18 90 160 150 38, head 100, direction Left. Expected order: 100, 90, 58, 55, 39, 38, 18, 150, 160. Total = (100 - 18) + (160 - 18) = 224.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int size, n, i, k, head, dir, temp, cur, total = 0;
    int rq[100];

    printf("Enter total number of disk blocks: ");
    scanf("%d", &size);
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    printf("Enter disk request string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &rq[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)              /* sort ascending */
        for (k = 0; k < n - i - 1; k++)
            if (rq[k] > rq[k + 1])
            {
                temp = rq[k]; rq[k] = rq[k + 1]; rq[k + 1] = temp;
            }

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0)
    {
        for (i = n - 1; i >= 0; i--)         /* left side first */
            if (rq[i] <= head)
            {
                printf(" -> %d", rq[i]); total += abs(cur - rq[i]); cur = rq[i];
            }
        for (i = 0; i < n; i++)              /* reverse at last request */
            if (rq[i] > head)
            {
                printf(" -> %d", rq[i]); total += abs(cur - rq[i]); cur = rq[i];
            }
    }
    else
    {
        for (i = 0; i < n; i++)              /* right side first */
            if (rq[i] >= head)
            {
                printf(" -> %d", rq[i]); total += abs(cur - rq[i]); cur = rq[i];
            }
        for (i = n - 1; i >= 0; i--)         /* reverse at last request */
```

```
            if (rq[i] < head)
            {
                printf(" -> %d", rq[i]); total += abs(cur - rq[i]); cur = rq[i];
            }
    }
    printf("\nTotal head movement = %d\n", total);
    return 0;
}
```

**Viva questions – LOOK** (also revise the disk questions in Slip 2)

1. What is LOOK? Like SCAN, but the head goes only as far as the last request in each direction.
2. Why is LOOK better than SCAN? It avoids useless movement to the disk edge.
3. What is C-LOOK? After the last request, the head jumps to the farthest request on the other side and continues in the same direction.
4. Show the manual calculation. (100 - 18) + (160 - 18) = 82 + 142 = 224.
5. Why two loops? One serves the chosen direction, the other serves the requests after reversing.
6. Why keep head and cur separately? head splits left and right requests, cur tracks the moving position.
7. Is the disk size used? It is only accepted as input. LOOK never needs to travel to the disk end.

## Slip 11

### Q.1 Banker's Algorithm – Menu Driven

Same program as Slip 1 Q.1. Use options in the order 1, 2, 4, 3, 5.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10

int max[MAX][MAX], need[MAX][MAX], allocation[MAX][MAX];
int available[MAX];
int np, nr;

void accept_matrix(int A[MAX][MAX])
{
    int i, j;
    for (i = 0; i < np; i++)
    {
        printf("P%d: ", i);
        for (j = 0; j < nr; j++)
            scanf("%d", &A[i][j]);
    }
}

void accept_vector(int A[MAX])
{
    int i;
    for (i = 0; i < nr; i++)
        scanf("%d", &A[i]);
}
```

```
void calculate_need()
{
    int i, j;
    for (i = 0; i < np; i++)
        for (j = 0; j < nr; j++)
            need[i][j] = max[i][j] - allocation[i][j];
    printf("\nNeed matrix calculated successfully.\n");
}

void display()
{
    int i, j;
    printf("\nProcess\tAllocation\tMax\t\tNeed\n");
    for (i = 0; i < np; i++)
    {
        printf("P%d\t", i);
        for (j = 0; j < nr; j++) printf("%d ", allocation[i][j]);
        printf("\t\t");
        for (j = 0; j < nr; j++) printf("%d ", max[i][j]);
        printf("\t\t");
        for (j = 0; j < nr; j++) printf("%d ", need[i][j]);
        printf("\n");
    }
    printf("\nAvailable: ");
    for (i = 0; i < nr; i++) printf("%d ", available[i]);
    printf("\n");
}

int main()
{
    int choice;
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter number of resources: ");
    scanf("%d", &nr);

    do
    {
        printf("\n----- MENU -----\n");
        printf("1. Accept Allocation\n2. Accept Max\n3. Calculate Need\n");
        printf("4. Accept Available\n5. Display Matrices\n6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1: printf("Enter Allocation Matrix:\n"); accept_matrix(allocation);
break;
            case 2: printf("Enter Max Matrix:\n"); accept_matrix(max); break;
            case 3: calculate_need(); break;
            case 4: printf("Enter Available Vector:\n"); accept_vector(available);
break;
            case 5: display(); break;
            case 6: break;
            default: printf("Invalid choice\n");
        }
```

```
    } while (choice != 6);
    return 0;
}
```

**Viva questions:** use the Banker's questions in Slip 1 Q.1.

### Q.2 Optimal Page Replacement

The original copy in this slip had two typos (print instead of printf, and fund instead of found). This corrected version is the same as Slip 4 Q.1. With 3 frames, total page faults = 9.

```
#include <stdio.h>

int main()
{
    int no_of_frames, no_of_pages, frames[10], pages[30], temp[10];
    int found, freeSlot, neverUsed, i, j, k, pos, max, faults = 0;

    printf("Enter number of frames: ");
    scanf("%d", &no_of_frames);
    printf("Enter number of pages: ");
    scanf("%d", &no_of_pages);
    printf("Enter page reference string: ");
    for (i = 0; i < no_of_pages; ++i)
        scanf("%d", &pages[i]);

    for (i = 0; i < no_of_frames; ++i)
        frames[i] = -1;

    printf("\nPage Replacement Process:\n");
    for (i = 0; i < no_of_pages; ++i)
    {
        found = freeSlot = 0;
        for (j = 0; j < no_of_frames; ++j)
            if (frames[j] == pages[i]) { found = 1; break; }

        if (!found)
        {
            for (j = 0; j < no_of_frames; ++j)
                if (frames[j] == -1)
                {
                    frames[j] = pages[i];
                    faults++;
                    freeSlot = 1;
                    break;
                }
        }

        if (!found && !freeSlot)
        {
            for (j = 0; j < no_of_frames; ++j)
            {
                temp[j] = -1;
                for (k = i + 1; k < no_of_pages; ++k)
                    if (frames[j] == pages[k]) { temp[j] = k; break; }
            }
```

```
            neverUsed = 0;
            for (j = 0; j < no_of_frames; ++j)
                if (temp[j] == -1) { pos = j; neverUsed = 1; break; }
            if (!neverUsed)
            {
                max = temp[0]; pos = 0;
                for (j = 1; j < no_of_frames; ++j)
                    if (temp[j] > max) { max = temp[j]; pos = j; }
            }
            frames[pos] = pages[i];
            faults++;
        }

        printf("Page %2d -> ", pages[i]);
        for (j = 0; j < no_of_frames; ++j)
        {
            if (frames[j] != -1) printf("%d ", frames[j]);
            else printf("- ");
        }
        printf("%s\n", found ? "(Hit)" : "(Fault)");
    }
    printf("\nTotal Page Faults = %d\n", faults);
    printf("Total Page Hits = %d\n", no_of_pages - faults);
    return 0;
}
```

**Viva questions:** use the page replacement questions in Slip 4 Q.1, plus:

1. Trace the string by hand with 3 frames. Faults occur at 8, 5, 7, 2, 3, 9, 4, 6, 2, giving 9 faults and 6 hits.
2. When several pages are never used again, which is replaced? Any of them is valid. The program picks the first.

## Slip 12

### Q.1 FCFS Scheduling

Accepts arrival time and first CPU burst, serves processes in order of arrival, and generates the next CPU burst randomly.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int n, i, j, t = 0, temp;
    int p[10], at[10], bt[10], st[10], ct[10], tat[10], wt[10];
    float twt = 0, ttat = 0;

    srand(time(NULL));
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
```

```
    {
        p[i] = i;
        printf("Arrival time and first CPU burst of P%d: ", i);
        scanf("%d %d", &at[i], &bt[i]);
    }

    for (i = 0; i < n - 1; i++)                /* sort by arrival time */
        for (j = 0; j < n - i - 1; j++)
            if (at[j] > at[j + 1])
            {
                temp = at[j]; at[j] = at[j + 1]; at[j + 1] = temp;
                temp = bt[j]; bt[j] = bt[j + 1]; bt[j + 1] = temp;
                temp = p[j];  p[j]  = p[j + 1];  p[j + 1]  = temp;
            }

    for (i = 0; i < n; i++)
    {
        if (t < at[i]) t = at[i];              /* CPU idle until arrival */
        st[i] = t;
        t += bt[i];
        ct[i] = t;
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        twt += wt[i];
        ttat += tat[i];
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i], at[i], bt[i], ct[i], tat[i],
wt[i]);

    printf("\nGantt Chart\n");
    for (i = 0; i < n; i++) printf("|  P%d  ", p[i]);
    printf("|\n");
    for (i = 0; i < n; i++) printf("%-7d", st[i]);
    printf("%d\n", ct[n - 1]);

    printf("\nAverage Waiting Time = %.2f\n", twt / n);
    printf("Average Turnaround Time = %.2f\n", ttat / n);

    printf("\nNext CPU burst (generated randomly):\n");
    for (i = 0; i < n; i++)
        printf("P%d : %d\n", p[i], rand() % 10 + 1);
    return 0;
}
```

**Viva questions – FCFS** (also revise the CPU scheduling questions in Slip 4)

1. What is FCFS? Processes run in order of arrival, like a FIFO queue.
2. Is FCFS preemptive? No.
3. What is the convoy effect? Short processes wait behind one long process, lowering CPU and I/O utilisation.
4. What is FCFS's advantage? It is simple, with no starvation.
5. What is its disadvantage? High average waiting time that depends on arrival order.

6. Why sort by arrival time? FCFS must serve the earliest arrival first.
7. What does if (t < at[i]) t = at[i] do? It handles idle CPU time when the next process has not arrived yet.
8. Why swap p[] along with at[] and bt[]? To keep each process number with its own times after sorting.

### Q.2 Fork – Parent Bubble Sort, Child Insertion Sort

The parent sorts with bubble sort and then waits for the child. The child sorts its own copy with insertion sort.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void bubble_sort(int a[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1])
            {
                temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
            }
}

void insertion_sort(int a[], int n)
{
    int i, j, temp;
    for (i = 1; i < n; i++)
    {
        temp = a[i];
        for (j = i - 1; j >= 0 && a[j] > temp; j--)
            a[j + 1] = a[j];
        a[j + 1] = temp;
    }
}

int main()
{
    int i, n, arr[30];
    pid_t pid;

    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    fflush(stdout);                 /* avoid duplicated output after fork */
    pid = fork();
```

```
    if (pid == 0)                   /* child */
    {
        printf("\nChild: PID = %d, Parent PID = %d\n", getpid(), getppid());
        insertion_sort(arr, n);
        printf("Child (Insertion Sort):");
        for (i = 0; i < n; i++) printf(" %d", arr[i]);
        printf("\n");
        exit(0);
    }
    else if (pid > 0)               /* parent */
    {
        printf("\nParent: PID = %d, Parent's parent PID = %d\n", getpid(),
getppid());
        bubble_sort(arr, n);
        printf("Parent (Bubble Sort):");
        for (i = 0; i < n; i++) printf(" %d", arr[i]);
        printf("\n");
        wait(NULL);
        printf("Child %d finished. Parent exiting.\n", pid);
    }
    else
        printf("Fork failed\n");
    return 0;
}
```

**Viva questions – fork and sorting**

1. What does fork() return? 0 in the child, the child's PID in the parent, and -1 on error.
2. Do parent and child share the array? No. The child gets its own copy (copy-on-write), so each sorts separately.
3. What do getpid() and getppid() return? The process's own ID and its parent's ID.
4. Why call wait()? The parent waits for the child to finish and collects its status, preventing a zombie.
5. How does wait() differ from waitpid()? wait waits for any child. waitpid waits for a specific child and supports options like WNOHANG.
6. Why is the output order unpredictable? Parent and child run concurrently and the scheduler decides who runs first.
7. What is the complexity of both sorts? O(n x n) in the worst case. Insertion sort is O(n) on sorted data.
8. How does bubble sort work? Adjacent elements are swapped so the largest moves to the end each pass.
9. How does insertion sort work? Each element is inserted into its correct place in the sorted left part.
10. What are zombie and orphan processes? A zombie has finished but was not waited for. An orphan's parent died first.
11. What is copy-on-write? Pages are shared until one process writes, and only then is a copy made.
12. Why fflush(stdout) before fork? Unflushed buffered output would be copied to the child and printed twice.
13. Name the exec family. execl, execv, execlp, execvp, execle and execve.

## Slip 13

### Q.1 Non-preemptive SJF Scheduling

Same program as Slip 4 Q.2.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int n, i, t = 0, done = 0, k = 0, idx;
    int at[10], bt[10], ct[10], tat[10], wt[10], fin[10] = {0};
    int order[10], st[10];
    float twt = 0, ttat = 0;

    srand(time(NULL));
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("Arrival time and first CPU burst of P%d: ", i);
        scanf("%d %d", &at[i], &bt[i]);
    }

    while (done < n)
    {
        idx = -1;
        for (i = 0; i < n; i++)
            if (!fin[i] && at[i] <= t)
                if (idx == -1 || bt[i] < bt[idx] || (bt[i] == bt[idx] && at[i] <
at[idx]))
                    idx = i;
        if (idx == -1) { t++; continue; }

        st[k] = t;
        order[k++] = idx;
        t += bt[idx];
        ct[idx] = t;
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];
        twt += wt[idx];
        ttat += tat[idx];
        fin[idx] = 1;
        done++;
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i, at[i], bt[i], ct[i], tat[i], wt[i]);

    printf("\nGantt Chart\n");
    for (i = 0; i < k; i++) printf("|  P%d  ", order[i]);
    printf("|\n");
```

```
    for (i = 0; i < k; i++) printf("%-7d", st[i]);
    printf("%d\n", t);

    printf("\nAverage Waiting Time = %.2f\n", twt / n);
    printf("Average Turnaround Time = %.2f\n", ttat / n);

    printf("\nNext CPU burst (generated randomly):\n");
    for (i = 0; i < n; i++)
        printf("P%d : %d\n", i, rand() % 10 + 1);
    return 0;
}
```

**Viva questions:** use the CPU scheduling and SJF questions in Slip 4 Q.2.

### Q.2 LOOK Disk Scheduling

Input: 15 30 55 90 125 140 170, head 100, direction Left. Expected order: 100, 90, 55, 30, 15, 125, 140, 170. Total = (100 - 15) + (170 - 15) = 85 + 155 = 240.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int size, n, i, k, head, dir, temp, cur, total = 0;
    int rq[100];

    printf("Enter total number of disk blocks: ");
    scanf("%d", &size);
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    printf("Enter disk request string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &rq[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (k = 0; k < n - i - 1; k++)
            if (rq[k] > rq[k + 1])
            {
                temp = rq[k]; rq[k] = rq[k + 1]; rq[k + 1] = temp;
            }

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0)
    {
        for (i = n - 1; i >= 0; i--)
            if (rq[i] <= head)
            {
                printf(" -> %d", rq[i]); total += abs(cur - rq[i]); cur = rq[i];
            }
        for (i = 0; i < n; i++)
```

```
            if (rq[i] > head)
            {
                printf(" -> %d", rq[i]); total += abs(cur - rq[i]); cur = rq[i];
            }
    }
    else
    {
        for (i = 0; i < n; i++)
            if (rq[i] >= head)
            {
                printf(" -> %d", rq[i]); total += abs(cur - rq[i]); cur = rq[i];
            }
        for (i = n - 1; i >= 0; i--)
            if (rq[i] < head)
            {
                printf(" -> %d", rq[i]); total += abs(cur - rq[i]); cur = rq[i];
            }
    }
    printf("\nTotal head movement = %d\n", total);
    return 0;
}
```

**Viva questions:** use the LOOK questions in Slip 5 Q.2 and the disk questions in Slip 2, plus:

1. What would SCAN give for this data? 100 + 170 = 270, because the head also travels to 0. LOOK saves 30 cylinders here.

## Slip 16

### Q.1 LRU Page Replacement (Counter Method)

Reference string: 3 5 7 2 5 1 2 3 1 3 5 3 1 6 2. With 3 frames, total page faults = 9.

```
#include <stdio.h>
#define MAX 20

int frames[MAX], ref[MAX], mem[MAX][MAX], time[MAX];
int faults, sp, m, n;

void accept()
{
    int i;
    printf("Enter no. of frames: ");
    scanf("%d", &n);
    printf("Enter no. of references: ");
    scanf("%d", &m);
    printf("Enter reference string:\n");
    for (i = 0; i < m; i++)
    {
        printf("[%d]=", i);
        scanf("%d", &ref[i]);
    }
}

void disp()
```

```
{
    int i, j;
    printf("\nReference string:\n");
    for (i = 0; i < m; i++) printf("%3d", ref[i]);
    printf("\n\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
        {
            if (mem[i][j] != -1) printf("%3d", mem[i][j]);
            else printf("   ");
        }
        printf("\n");
    }
    printf("\nTotal Page Faults: %d\n", faults);
}

int search(int pno)
{
    int i;
    for (i = 0; i < n; i++)
        if (frames[i] == pno) return i;
    return -1;
}

int get_lru()                       /* frame with the oldest last-use time */
{
    int i, min_i = 0, min = 9999;
    for (i = 0; i < n; i++)
        if (time[i] < min) { min = time[i]; min_i = i; }
    return min_i;
}

void lru()
{
    int i, j, k;
    for (i = 0; i < m && sp < n; i++)       /* fill empty frames */
    {
        k = search(ref[i]);
        if (k == -1)
        {
            frames[sp] = ref[i];
            time[sp] = i;
            faults++;
            sp++;
        }
        else
            time[k] = i;
        for (j = 0; j < n; j++) mem[j][i] = frames[j];
    }
    for (; i < m; i++)                       /* replacement phase */
    {
        k = search(ref[i]);
        if (k == -1)
        {
```

```
            sp = get_lru();
            frames[sp] = ref[i];
            time[sp] = i;
            faults++;
        }
        else
            time[k] = i;                    /* update last-use time */
        for (j = 0; j < n; j++) mem[j][i] = frames[j];
    }
}

int main()
{
    int i, j;
    for (i = 0; i < MAX; i++)
    {
        frames[i] = -1;
        time[i] = 0;
        for (j = 0; j < MAX; j++) mem[i][j] = -1;
    }
    faults = sp = 0;
    accept();
    lru();
    disp();
    return 0;
}
```

**Viva questions – LRU** (also revise the page replacement questions in Slip 4)

1. What is LRU? It replaces the page not used for the longest time in the past.
2. What is the counter method? Each frame stores the time of its last use. The smallest time is the LRU page.
3. What is the other way to implement LRU? A stack: a referenced page moves to the top, and the bottom page is the LRU.
4. What does get_lru() return? The index of the frame with the minimum time value.
5. Why update time[k] on a hit? The page was just used, so its last-use time must be refreshed.
6. What is sp? The next empty frame while filling, then the frame chosen for replacement.
7. What is mem[][]? The frame contents after each reference, used for the display table.
8. Does LRU suffer from Belady's anomaly? No. It is a stack algorithm.
9. How is LRU related to Optimal? LRU approximates Optimal using the past instead of the future.
10. What hardware support does LRU need? A counter or clock per page, or reference bits, which is costly.
11. What are LRU approximations? The reference bit and the second-chance (clock) algorithm.
12. Trace the string by hand. Faults at 3, 5, 7, 2, 1, 3, 5, 6, 2 give 9 faults and 6 hits.

## Q.2 Shell with the count command

Same program as Slip 1 Q.2.

```
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void make_toks(char *s, char *tok[])
{
    int i = 0;
    char *p = strtok(s, " \t");
    while (p != NULL)
    {
        tok[i++] = p;
        p = strtok(NULL, " \t");
    }
    tok[i] = NULL;
}

void count(char *fn, char op)
{
    int fh, cc = 0, wc = 0, lc = 0, inword = 0;
    char c;
    fh = open(fn, O_RDONLY);
    if (fh == -1)
    {
        printf("File %s not found.\n", fn);
        return;
    }
    while (read(fh, &c, 1) > 0)
    {
        cc++;
        if (c == '\n') lc++;
        if (c == ' ' || c == '\t' || c == '\n') inword = 0;
        else if (!inword) { inword = 1; wc++; }
    }
    close(fh);
    switch (op)
    {
        case 'c': printf("No. of characters: %d\n", cc); break;
        case 'w': printf("No. of words: %d\n", wc); break;
        case 'l': printf("No. of lines: %d\n", lc); break;
        default : printf("Invalid option. Use c, w or l\n");
    }
}

int main()
{
    char buff[80], *args[10];
    pid_t pid;

    while (1)
    {
        printf("$ ");
        fflush(stdout);
        if (fgets(buff, 80, stdin) == NULL) break;
        buff[strcspn(buff, "\n")] = '\0';
        make_toks(buff, args);
```

```
        if (args[0] == NULL) continue;

        if (strcmp(args[0], "quit") == 0)
            break;
        else if (strcmp(args[0], "count") == 0)
        {
            if (args[1] == NULL || args[2] == NULL)
                printf("Usage: count c|w|l filename\n");
            else
                count(args[2], args[1][0]);
        }
        else
        {
            pid = fork();
            if (pid > 0)
                wait(NULL);
            else if (pid == 0)
            {
                if (execvp(args[0], args) == -1)
                {
                    printf("Bad command.\n");
                    exit(1);
                }
            }
            else
                printf("Fork failed\n");
        }
    }
    return 0;
}
```

**Viva questions:** use the shell questions in Slip 1 Q.2.

## Slip 25

### Q.1 Orphan Process

The parent creates a child and terminates first. After sleeping, the child prints a different parent PID, showing it was adopted.

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid > 0)                    /* parent */
    {
        printf("I am PARENT, my PID = %d, my parent's PID = %d\n", getpid(),
getppid());
        sleep(2);                   /* let the child print its first parent PID */
        printf("Parent terminating...\n");
    }
```

```
    else if (pid == 0)              /* child */
    {
        printf("I am CHILD, my PID = %d, my parent's PID = %d\n", getpid(),
getppid());
        sleep(10);                  /* parent finishes in the meantime */
        printf("\nAfter 10 seconds - child is now ORPHAN\n");
        printf("I am CHILD, my PID = %d, my new parent's PID = %d\n", getpid(),
getppid());
    }
    else
        printf("Failed to create child process\n");
    return 0;
}
```

**Viva questions – Orphan process**

1. What is an orphan process? A child still running after its parent has terminated.
2. Who adopts an orphan? init or systemd (PID 1), or a subreaper process, so the new PPID may not be 1 on Ubuntu.
3. Why sleep(10) in the child? To keep the child alive after the parent exits.
4. Why sleep(2) in the parent? So the child prints its original parent PID before the parent exits.
5. How do you know the child became an orphan? getppid() returns a different value after the sleep.
6. How does an orphan differ from a zombie? An orphan is running with a dead parent. A zombie has finished but was not reaped.
7. How do you create a zombie? The child exits while the parent sleeps without calling wait().
8. Is an orphan harmful? No. init adopts and reaps it. Zombies waste process table entries.
9. Why does the shell prompt appear before the child's last output? The shell waits only for the parent, which has exited.
10. What is a daemon? A background process that is deliberately orphaned and detached from the terminal.
11. Which commands show processes? ps -el, ps -ef, top and pstree.
12. What does sleep() do? It suspends the process for the given seconds. It is declared in unistd.h.
13. What is pid_t? The data type for process IDs, defined in sys/types.h.

## Q.2 Shell with the search command

Supports search f (first occurrence), search a (all occurrences) and search c (count of occurrences).

```
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void make_toks(char *s, char *tok[])
{
    int i = 0;
    char *p = strtok(s, " \t");
    while (p != NULL)
```

```
    {
        tok[i++] = p;
        p = strtok(NULL, " \t");
    }
    tok[i] = NULL;
}

void search(char *fn, char op, char *pattern)
{
    int fh, count = 0, line = 0, j = 0, n, done = 0;
    char buff[256], c, *p;

    fh = open(fn, O_RDONLY);
    if (fh == -1)
    {
        printf("File %s not found\n", fn);
        return;
    }
    if (op != 'f' && op != 'a' && op != 'c')
    {
        printf("Invalid option. Use f, a or c\n");
        close(fh);
        return;
    }

    while (!done)
    {
        n = read(fh, &c, 1);
        if (n > 0 && c != '\n' && j < 255)
        {
            buff[j++] = c;                 /* build one line */
            continue;
        }
        if (n <= 0 && j == 0) break;       /* end of file */
        buff[j] = '\0';
        j = 0;
        line++;
        if (n <= 0) done = 1;              /* last line without newline */

        if (op == 'c')
        {
            p = buff;
            while ((p = strstr(p, pattern)) != NULL)
            {
                count++;
                p += strlen(pattern);
            }
        }
        else if (strstr(buff, pattern))
        {
            printf("%d: %s\n", line, buff);
            count++;
            if (op == 'f') break;          /* first occurrence only */
        }
    }
```

```
    close(fh);

    if (op == 'c')
        printf("Total no. of occurrences = %d\n", count);
    else if (count == 0)
        printf("Pattern not found\n");
}

int main()
{
    char buff[80], *args[10];
    pid_t pid;

    while (1)
    {
        printf("$ ");
        fflush(stdout);
        if (fgets(buff, 80, stdin) == NULL) break;
        buff[strcspn(buff, "\n")] = '\0';
        make_toks(buff, args);
        if (args[0] == NULL) continue;

        if (strcmp(args[0], "quit") == 0)
            break;
        else if (strcmp(args[0], "search") == 0)
        {
            if (args[1] == NULL || args[2] == NULL || args[3] == NULL)
                printf("Usage: search f|a|c filename pattern\n");
            else
                search(args[2], args[1][0], args[3]);
        }
        else
        {
            pid = fork();
            if (pid > 0)
                wait(NULL);
            else if (pid == 0)
            {
                if (execvp(args[0], args) == -1)
                {
                    printf("Bad command.\n");
                    exit(1);
                }
            }
            else
                printf("Fork failed\n");
        }
    }
    return 0;
}
```

**Viva questions – search** (also revise the shell questions in Slip 1 Q.2)

1. What does strstr() do? It finds the first occurrence of a substring and returns a pointer to it, or NULL.

2. What do f, a and c do? f prints the first matching line, a prints all matching lines, and c counts total occurrences.
3. Which Linux command is similar? grep (grep -m1 for first, grep -c to count matching lines).
4. Why read one character at a time? To build each line until a newline is found.
5. Why p += strlen(pattern) in count mode? To continue searching after the current match without counting overlaps.
6. How would you make the search case-insensitive? Use strcasestr() or convert both strings to lowercase.
7. Why print the line number? To show where the pattern occurs in the file.
8. How is the last line without a newline handled? When read() returns 0, the partly built line is still searched.
9. Why the j < 255 check? To prevent writing past the end of buff.

## Slip 28

### Q.1 SCAN Disk Scheduling

Input: 10 25 75 90 130 145 180 55, head 80, direction Left, disk size 200. Expected order: 80, 75, 55, 25, 10, 0, 90, 130, 145, 180. Total = 80 + 180 = 260.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int rq[100], n, i, j, temp, size, head, dir, idx, cur, total = 0;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &size);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string:\n");
    for (i = 0; i < n; i++) scanf("%d", &rq[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (rq[j] > rq[j + 1])
            {
                temp = rq[j]; rq[j] = rq[j + 1]; rq[j + 1] = temp;
            }

    idx = n;
    for (i = 0; i < n; i++)
        if (rq[i] > head) { idx = i; break; }

    cur = head;
    printf("\nOrder of service: %d", head);
```

```
    if (dir == 0)
    {
        for (i = idx - 1; i >= 0; i--)
        {
            printf(" -> %d", rq[i]); total += cur - rq[i]; cur = rq[i];
        }
        printf(" -> 0"); total += cur; cur = 0;
        for (i = idx; i < n; i++)
        {
            printf(" -> %d", rq[i]); total += rq[i] - cur; cur = rq[i];
        }
    }
    else
    {
        for (i = idx; i < n; i++)
        {
            printf(" -> %d", rq[i]); total += rq[i] - cur; cur = rq[i];
        }
        printf(" -> %d", size - 1); total += size - 1 - cur; cur = size - 1;
        for (i = idx - 1; i >= 0; i--)
        {
            printf(" -> %d", rq[i]); total += cur - rq[i]; cur = rq[i];
        }
    }
    printf("\nTotal head movement = %d\n", total);
    return 0;
}
```

**Viva questions:** use the SCAN questions in Slip 3 Q.2 and the disk questions in Slip 2, plus:

1. Show the manual calculation. (80 - 0) + (180 - 0) = 260.
2. What would LOOK give for this data? (80 - 10) + (180 - 10) = 240.

### Q.2 MFU Page Replacement

Reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2. With 3 frames, total page faults = 10 (ties broken in FIFO order).

```
#include <stdio.h>
#define MAX 20

int frames[MAX], ref[MAX], mem[MAX][MAX], count[MAX];
int faults, sp, m, n;

void accept()
{
    int i;
    printf("Enter no. of frames: ");
    scanf("%d", &n);
    printf("Enter no. of references: ");
    scanf("%d", &m);
    printf("Enter reference string:\n");
    for (i = 0; i < m; i++)
    {
        printf("[%d]=", i);
        scanf("%d", &ref[i]);
```

```
    }
}

void disp()
{
    int i, j;
    printf("\nReference string:\n");
    for (i = 0; i < m; i++) printf("%3d", ref[i]);
    printf("\n\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
        {
            if (mem[i][j] != -1) printf("%3d", mem[i][j]);
            else printf("   ");
        }
        printf("\n");
    }
    printf("\nTotal Page Faults: %d\n", faults);
}

int search(int pno)
{
    int i;
    for (i = 0; i < n; i++)
        if (frames[i] == pno) return i;
    return -1;
}

int get_mfu(int start)              /* highest count; ties go in FIFO order */
{
    int i = start, max_i = start, max = -1;
    do
    {
        if (count[i] > max) { max = count[i]; max_i = i; }
        i = (i + 1) % n;
    } while (i != start);
    return max_i;
}

void mfu()
{
    int i, j, k;
    for (i = 0; i < m && sp < n; i++)       /* fill empty frames */
    {
        k = search(ref[i]);
        if (k == -1)
        {
            frames[sp] = ref[i];
            count[sp] = 1;
            faults++;
            sp++;
        }
        else
            count[k]++;
```

```
        for (j = 0; j < n; j++) mem[j][i] = frames[j];
    }
    sp = 0;
    for (; i < m; i++)                      /* replacement phase */
    {
        k = search(ref[i]);
        if (k == -1)
        {
            sp = get_mfu(sp);
            frames[sp] = ref[i];
            count[sp] = 1;                  /* new page starts with count 1 */
            faults++;
            sp = (sp + 1) % n;
        }
        else
            count[k]++;
        for (j = 0; j < n; j++) mem[j][i] = frames[j];
    }
}

int main()
{
    int i, j;
    for (i = 0; i < MAX; i++)
    {
        frames[i] = -1;
        count[i] = 0;
        for (j = 0; j < MAX; j++) mem[i][j] = -1;
    }
    faults = sp = 0;
    accept();
    mfu();
    disp();
    return 0;
}
```

**Viva questions – MFU** (also revise the page replacement questions in Slip 4)

1. What is MFU? Most Frequently Used: it replaces the page with the highest reference count.
2. What is the logic behind MFU? A page with a high count has probably finished its work. A page with a low count was just brought in and will be needed soon.
3. What is LFU? Least Frequently Used: it replaces the page with the lowest count, the opposite of MFU.
4. Are MFU and LFU common in real systems? No. They are costly and do not approximate Optimal well.
5. What is count[]? The reference frequency of the page in each frame.
6. Why reset count to 1 on replacement? It is a new page, so the old page's history does not apply.
7. How are ties broken? get_mfu() scans from sp and takes the first maximum, which gives FIFO order.
8. Why sp = (sp + 1) % n? To move the tie-break pointer circularly.
9. Compare faults for this string with 3 frames. Optimal gives 9, MFU gives 10.
10. Does MFU suffer from Belady's anomaly? It can, because it is not a stack algorithm.